

STAPLE Documentation

Erin M. Buchanan

August 31, 2026

Table of contents

1	Preface	4
2	Introduction	6
3	Core Concepts	8
3.1	STAPLE Elements	8
3.2	Metadata	8
3.3	Definitions of Terms	9
3.3.1	User	9
3.3.2	Main and Project Level Options	10
3.3.3	Project Level Options	13
4	Metadata and Forms	15
4.1	What Is Metadata?	15
4.2	Why Metadata Matters	15
4.3	Forms: How STAPLE Collects Metadata	16
4.4	Building a Form	16
4.5	Item Types and Rules	22
4.5.1	Dependencies	23
4.6	Using a Form in a Task	24
4.7	Where Form Data Lives Afterward	29
5	Installation Guide	31
5.1	Assumptions	31
5.2	Local Development or Hosting	31
5.3	Installation Steps	32
5.3.1	Web Server Installation	32
5.3.2	Blitz/JS Installation	32
5.3.3	Clone the Repository	33
5.3.4	Database Installation	34
5.3.5	Connect Database to STAPLE App	35
5.3.6	Configure Email	37
5.3.7	Install STAPLE Requirements	38
5.3.8	Setting Up the STAPLE Viewer Worker	39
5.3.9	Starting the App - Local Testing	41
5.3.10	Starting the App - Production	41

5.3.11	Keeping the App Going	42
5.3.12	Setting Up the Proxy (Nginx)	43
5.3.13	Enable HTTPS	44
5.3.14	Install pgAdmin (Optional)	44
5.3.15	Updating STAPLE	50
5.4	Common Issues & Fixes	51
5.4.1	Prisma / Database Errors	51
5.4.2	Build Errors	52
5.4.3	Email Issues	52
5.4.4	Nginx / Proxy Issues	52
5.4.5	Service Issues	53
5.4.6	General Tips	53
6	Glossary	54
7	References	55

1 Preface

Scientific research has become increasingly complex, requiring specialized skills, interdisciplinary work, and collaboration among large teams. Managing such projects and tracking data and metadata has become a significant challenge. The need for FAIR (findable, accessible, interoperable, and reusable Wilkinson et al., 2016) open data, materials, and metadata has similarly grown, especially considering recent mandates for required sharing for federally funded projects (Rep. Foxx, 2019; Rep. Ryan, 2019). While user-friendly software tools that help researchers structure metadata exist, few researchers are aware of their necessity and usefulness, the onus is still on the project team to collect and curate this information. Studies using teams of researchers require sophisticated tracking of all project elements, and therefore, there is a need for scientific project management software that is tailored to the management of all manner of science projects. Software that promotes best practices in FAIR data and metadata would increase openness in all reporting, and tracking, and reporting standards.

Currently, the tools and websites designed for researchers are focused on *getting* researchers to share their materials, code, and data (i.e., [Open Science Framework](#), [FigShare](#), [Zenodo](#)). These repositories represent a necessary resource for long-term storage of outputs, but do not help researchers organize or track information during the life of a study. Project management software, like Asana, Monday, or ClickUp, are designed from a business perspective that is not tailored to scientific research. While the features that scientific research requires may be found in some individual project management software, no current solution provides all the essential features, such as the ability to assign tasks at different scales (e.g., teams, individuals), integrated metadata, fully transparent access to all information in the project manager, and long-term storage that complies with international data privacy regulations. Project management software is designed to *get things done* rather than *document* the way a project was completed, so attempts to use existing software for these features often involve ‘hacking’ it to extract necessary records.

To this end, we developed STAPLE (**S**cience **T**racking **A**cross the **P**roject **L**ifEspan), which not only helps with the unique challenges of project management of research but includes open and transparent documentation of data and metadata, as well as providing templates for minimum metadata collection. STAPLE allows users to add project components based on research type, assign timelines, delegate tasks to individuals or groups, and link to long-term storage of research outputs. STAPLE helps track authorship credit and contributor roles via tasks or contributors. STAPLE includes default metadata standards for documenting common research outputs and is fully open-source, enabling community input and adaptation. Designed for global reach, STAPLE supports the transition to open science by including researchers and

outputs that are often overlooked, regardless of technical expertise. Through its point-and-click interface, and downloadable documentation outputs, STAPLE makes it easier for all researchers to participate in transparent, inclusive, and reproducible science.

2 Introduction

The STAPLE mission: *Project Management* software that allows you to *document* your research project to improve *transparency*.

As researchers, we must balance attention to the big picture: the overarching goals, timelines, theories, and hypotheses of a project, with the smaller goals and tasks required to realize those ambitions. While most researchers receive training in designing studies, collecting and analyzing data, and publishing findings, formal instruction in project management is rare. Instead, these skills are often acquired informally, modeled within the research lab environment and shaped by a supervisor's style. Each project also introduces unique needs and circumstances, and even the addition of a single collaborator can generate new layers of coordination and complexity.

Evidence indicates that the number of co-authors and collaborators on research projects has increased steadily over time (Wuchty et al., 2007). Advances in digital communication and collaboration tools such as Zoom, GitHub, and Google Workspace, combined with the globalization of research, have enabled teams to work together across great distances and disciplinary boundaries (Cummings & Kiesler, 2005, 2007). Large-scale collaborations are increasingly encouraged, not only to tackle ambitious scientific questions, but also to enhance the diversity of participants, researchers, and cultures represented in data (Adan, 2023; Moshontz et al., 2018; Swartz et al., 2019). The inclusion of multiple collaborators, each embedded in different lab cultures, institutions, geopolitical contexts, and languages, increases the complexity of managing research projects.

So why do we need software?

General project management platforms, such as Asana, Monday, ClickUp, and Trello, are widely used in business settings, but they were designed around business-oriented workflows and templates. Research projects, however, present unique challenges that do not map neatly onto these models. Scientific projects often require coordination of specialized tasks such as research design, ethical approval, data collection across multiple sites, metadata documentation, open data sharing, and publication requirements (Borghi & Gulick, 2021; Fecher & Friesike, 2014).

Large-scale collaborations have experimented with adapting existing structures, sometimes treating multi-team projects like classrooms or leveraging educational technologies, but these solutions are partial fits at best. What researchers need is a system that recognizes the scientific lifecycle: from ideas and preregistration, to data collection, to analysis, dissemination, and long-term archiving. STAPLE is designed for scientists, by scientists. It was built with input

not only from researchers but also from key stakeholders in the broader research ecosystem, including funders, librarians, and journals, ensuring that the software supports transparency, reproducibility, and effective project management across diverse scientific contexts.

So what's the deal with documentation?

In tandem with the encouragement for interdisciplinary, diverse teams, researchers have begun to focus on the improvement of science through transparency and sharing. The Transparency and Openness guidelines establish ideals for projects to be reproducible, open, and transparency through data sharing, code/analysis sharing, materials sharing, pre-registration, and replication (Grant et al., 2025). Journals require different levels of sharing for publication, which has pushed researchers to begin to implement these practices. However, the simple sharing of data, code, and materials does not make them FAIR: Findable, Accessible, Interoperable, and Reusable (Wilkinson et al., 2016). Simple sharing of data does not necessarily allow someone to find that data in relation to the research project, or allow a researcher to know what V1 means in the data, or even reuse part of the data.

Creating adequate documentation for the reuse of any output from a project can be difficult. There are often no clear standards, the work can be time consuming, and is often left to the end of a project when important details may have been forgotten. STAPLE is designed to encourage you to document information along the way and to provide guidance with what you should include with each document. As you complete specific task, such as create your materials for a study, you will be able to link your project to those materials and include information to interpret and understand those documents. We provide a set of minimum documentation standards for multiple types of documents - and the ability for you to include extra information if your field has set standards.

Ok, so that's transparent?

STAPLE allows you to track and run your project - and then include research outputs from that project. Each step of the way, STAPLE collects data about who did what and when. If you are the only person working on a project, it's simple: you will export a final project timeline that includes information about each step of the project, each output, and the documentation needed to make those outputs useful. If you work with other people, STAPLE will allow you to assign tasks to those individuals (or teams of individuals) to complete as part of the project management functionality. At the same time, STAPLE tracks their work and creates a project timeline that shows that they completed a task for the project.

This component of the software was designed to align with contributorship models of research - instead of "authorship", each person receives credit for the components they contributed to the project (Allen et al., 2014, 2019). Each field has different standards for authorship, and many individual's work may not be acknowledged due to those cultures. Using STAPLE, their work would be credited, even if they did not earn final "authorship" on a publication. This output is also useful for defining contributions into systems such as CRediT - where each person's role is binned into categories. By linking each role to specific tasks, we can transparently show what each role means to a specific project.

3 Core Concepts

In this section, we introduce the core terminology for STAPLE elements to help users understand their purpose and function. The following section will demonstrate how these elements appear in practice, with screenshots and step-by-step guides for using them within STAPLE.

3.1 STAPLE Elements

Research projects are made up of many moving parts: overarching goals, specific hypotheses, tasks to carry out, and the people who contribute along the way. To manage this complexity, STAPLE organizes projects into a small set of core elements. These elements provide a shared structure that scientists and collaborators can understand quickly, regardless of discipline.

At the highest level, a Project represents the scientific effort being undertaken: its aims, scope, and timeline. Within each project, researchers define Tasks, which capture the individual units of work needed to achieve those aims. Teams and Contributor/Members provide the human side of the system, reflecting roles, privileges, and collaborations across labs and institutions. Every action taken on a task, whether assigning it, updating its status, or marking it as complete, is recorded in Task Logs, creating a transparent history of progress. Finally, Tags allow projects to be categorized, searched, and documented in ways that make sense for science.

Together, these elements form the backbone of STAPLE, giving research teams a clear, consistent framework for planning and documenting their work.

3.2 Metadata

In scientific research, data alone is never enough, what makes data meaningful is the metadata that describes it. Metadata captures the critical context behind research activities: the methods used, the instruments applied, the variables measured, and the conditions under which data were collected. Without metadata, results can be difficult to reproduce, interpret, or build upon.

STAPLE treats metadata as a first-class feature, not an afterthought. The software also automatically records who did what, and when, embedding provenance metadata directly within projects. Project information, such as data, materials, and analysis code, can be linked within STAPLE and enriched with metadata. These details are captured using the Forms

feature, which provides structured templates tailored to research needs. Completion of metadata can be assigned to contributors via Tasks, making documentation part of the normal project workflow rather than an extra burden.

By embedding metadata capture directly into everyday research activities, STAPLE makes it easier for scientists to document their work transparently, meet the expectations of journals and funders, and create records that future collaborators, or even their own future selves, can interpret and reuse. All metadata can be exported as machine-readable JSON for computational use and as human-readable interactive Project Summary reports for sharing and communication.

3.3 Definitions of Terms

3.3.1 User

A user in STAPLE is anyone with an account who has accepted the platform's terms and conditions. Users can create projects and related elements within their own accounts, and they can also participate in projects owned by others. Users can log in and out, update their personal information (e.g., username, password, email, and profile details), and delete their account. They may add and manage contributors, organize teams within their projects, and assign tasks.

Note: Deleting your STAPLE account *does not* remove your past contributions to other projects. You will lose editing access, but only a project administrator can fully remove you from their project.

3.3.1.1 Project Administrator

- The user who creates a project is automatically assigned as its project administrator.
- Additional administrators can be added to a project as needed.
- Every project must have at least one administrator at all times.
- Administrators have the highest level of control, including managing contributors, editing project settings, and assigning roles.

3.3.1.2 Contributor

- A contributor is a user added to a project by an administrator.
- Contributors generally have limited access: they can view project information and complete tasks assigned to them but cannot change project settings or manage other users.
- This role allows project teams to include collaborators without granting full administrative privileges.

3.3.1.3 Teams

- Teams are a special type of contributor within a project.
- A user can belong to multiple teams in the same project.
- Teams are useful when a task requires collaboration by a group rather than an individual.

For example, a data collection team working with human participants may need to complete an ethics application. Only one person needs to submit the form within STAPLE, but the entire team receives credit for completing the task. Similarly, a translation team might be responsible for converting study materials from French into Spanish. The collective goal is to provide the final Spanish materials, but all members of the team are credited for the work.

3.3.2 Main and Project Level Options

3.3.2.1 Dashboard

The Dashboard is your personalized home page in STAPLE. It provides a high-level overview of all the projects you're involved in, along with recent updates and tasks assigned to you. From here, you can quickly jump into active projects, track progress at a glance, and see what needs your attention today. Think of it as your command center for research project management.

In addition to this main dashboard, each project also has its own Project Dashboard. While the main dashboard gives you a cross-project snapshot, the project dashboard focuses on the details of a single project, its contributors, tasks, metadata, milestones, and outputs. Together, the two levels of dashboards help you move easily between the big picture and the project-specific details.

3.3.2.2 Projects

Projects are the central containers for scientific work. Each project includes its own tasks, teams, metadata, and outputs. On the Projects page, you can:

- Create a new project.
- View and manage existing projects.
- Go into the project level dashboard.

Projects provide the big-picture structure that keeps research activities organized and transparent.

3.3.2.3 Invitations

Invitations are how collaborators join projects in STAPLE. Project administrators send invitations that specify the role a user will have once they accept.

- On the main dashboard, you can view all of your pending invitations across projects and choose to accept or decline them.
- On a project dashboard, only project administrators can see the invitations they have sent for that specific project.

A contributor cannot be assigned tasks or added to teams until they have accepted their invitation. This system ensures that every collaborator's involvement is explicit, documented, and consented to before work begins. Each person will receive an email from `app@staple-science.com`. The email may go to spam and may need to be marked as safe to ensure all invitation and notification emails are received. We also have emails that can be sent from `app@staplescience.com` to alleviate issues with universities blocking `.science` addresses.

3.3.2.4 Tasks

Tasks break a project down into actionable steps and are the backbone of daily research activity in STAPLE.

- On the main dashboard, the Tasks page shows a consolidated view of all tasks assigned to you across every project, so nothing slips through the cracks.
- On a project dashboard, project administrators can view and manage all tasks within that project, while contributors see only the tasks assigned to them.

Within a project, you can:

- Create new tasks for yourself or others.
- Assign tasks to contributors or teams.
- Update task status (to-do, in progress, complete).
- Add labels, metadata, and logs to track progress.

This two-level view ensures that contributors can stay focused on their own responsibilities while project managers maintain oversight of the entire project's workflow.

3.3.2.5 Notifications

Notifications keep you informed about project activity without needing to check each page manually. These alerts let you know when:

- You've been assigned a new task.

- A task you're involved in has been updated, completed, or approved.
- You've received a project invitation.
- You've been added to a team.

On the main dashboard, notifications show updates from all of your projects in one place. On a project dashboard, notifications are filtered to that project only, helping you focus on what matters most in the current context. The Notifications page ensures you never miss important updates in collaborative projects. Each day you will receive a summary email of your notifications from `app@staplescience.com`. The email may go to spam and may need to be marked as safe to ensure all notification emails are received. We also have emails that can be sent from `app@staplescience.com` to alleviate issues with universities blocking `.science` addresses.

3.3.2.6 Forms

Forms are structured templates for collecting metadata at the project or task level. They make sure essential details, such as ethics approvals, instrument calibration, dataset versions, or preregistration links, are captured consistently.

STAPLE includes a library of form templates, giving you a starting point if you're not sure what metadata to collect. You can also design your own forms tailored to your project's needs. Forms must be created before they can be used in a project, ensuring that the structure is in place before contributors start filling them out.

Once in use, forms become a powerful way to collect metadata directly within your workflow. You can:

- Fill out forms as part of project tasks.
- Assign forms to contributors or teams to ensure documentation responsibilities are shared.
- Reuse templates across multiple projects for consistency.

By embedding metadata capture directly into the research process, forms help projects meet transparency and reproducibility standards while reducing the burden of after-the-fact documentation.

3.3.2.7 Roles

Roles in STAPLE describe the kinds of contributions people make within a project. While they draw inspiration from frameworks like the CRediT taxonomy, they are not limited to authorship or publication contexts. Instead, roles provide a flexible way to document responsibilities and ensure that contributions are visible across the lifespan of a project.

In STAPLE, roles can be:

- Assigned to people → to show what responsibilities each contributor holds in a project.
- Assigned to tasks → to clarify what kind of contribution a task represents and how it fits into the project.

Roles provide a flexible system for understanding who did what and for standardizing contributions across projects. Role templates are available to add to your account, which you can use across all projects. They must be created on the main dashboard before you can use them.

3.3.3 Project Level Options

3.3.3.1 Tags

Tags are flexible labels that help organize and filter milestones, tasks, and contributors. Tags make it easier to search and group related work across large projects. You can view customized tracking by using tags across these parts of the project.

3.3.3.2 Milestones

Milestones mark major points of progress within a project and help teams track deadlines and achievements. In STAPLE, milestones can also group related tasks together, providing a clear structure for managing phases of a project. Each milestone is tied to a timeline, and STAPLE automatically visualizes them in a Gantt chart, making it easier to see how tasks and deadlines align. This helps teams monitor progress, anticipate bottlenecks, and keep projects moving forward on schedule.

3.3.3.3 Notes

Notes provide a space for free-text documentation within a project. Unlike structured metadata captured through Forms, Notes allow contributors to record context, reminders, or reflections in their own words.

Notes can be:

- Private – visible only to the author.
- Shared with contributors – if shared by a project administrator, contributors can view them.
- Restricted to administrators – for communication only among project managers.

This flexibility makes Notes useful for everything from personal reminders to collaborative discussions among administrators.

3.3.3.4 Form Data

Form Data is the collected information from metadata Forms within a project. It provides a centralized view of all structured metadata entries—such as instrument details, ethics protocols, or dataset versions—that have been completed by contributors. Form Data can be reviewed, exported, and updated if errors have occurred.

3.3.3.5 Summary

The Summary page generates a shareable overview of the project. It consolidates contributors, tasks, roles, metadata, milestones, and outputs into one report. Project Summaries can be exported in both machine-readable formats (JSON) and human-readable interactive reports, making them useful for collaboration, archiving, and compliance with funder or journal requirements.

3.3.3.6 Settings

The Settings page allows administrators to configure project-level metadata. This includes details such as the project title, description, timeline, and high-level information about the project's scope.

4 Metadata and Forms

The [Core Concepts](#) section introduced metadata and Forms as two of STAPLE’s core elements. This section goes one level deeper: what metadata actually is, why it is worth the extra effort to collect, and the concrete steps for building a form and attaching it to a Task so that metadata collection becomes part of your everyday project work.

4.1 What Is Metadata?

Metadata is *data about data* — information that describes a resource rather than being the resource itself. In a research project, “resource” covers more than a final dataset. It can include:

- **Documents:** ethics approvals, stimuli and materials, translations, notes, manuscripts.
- **Data:** primary data you collect (text, audio, video, images) and secondary data you reuse from elsewhere.
- **Code:** analysis scripts, study-implementation code, or software.
- **Project-level information:** the project itself, the people involved, their organizations and funders, and who contributed what.

For example, a dataset’s metadata might record its file format, where it can be downloaded, and who created it — without that information being part of the data file itself.

Metadata can be **human-readable** (a descriptive report or a simple table of resources and descriptions) or **machine-readable** (a structured format like JSON that a computer system can parse and index). STAPLE’s Forms produce both: a form you fill out in the app *and* a structured JSON record behind the scenes that can be exported and shared.

4.2 Why Metadata Matters

Metadata is often the difference between a resource that can be reused and one that quietly becomes unusable once the people who made it move on. The FAIR principles describe what good metadata makes possible (Wilkinson et al., 2016):

- **Findable:** resources can be discovered through databases, search engines, and repositories.

- **Accessible:** others know how to obtain or use the resource.
- **Interoperable:** the resource can be combined with other resources and tools.
- **Reusable:** enough context is provided that someone else (including future-you) can reuse it correctly.

Beyond FAIR, there are practical reasons to collect metadata as you go rather than after the fact:

- **It’s increasingly required.** Journals adopting the Transparency and Openness guidelines, and funders with open-data mandates, expect metadata alongside shared resources (Grant et al., 2025).
- **It supports reproducibility.** Collaborators and reviewers can independently verify and correct your work when the context behind it is documented.
- **It enables reuse and extension.** Well-described resources can be combined, reanalyzed, or extended in ways the original authors didn’t anticipate.
- **It’s easy to lose track of later.** Details like instrument settings, consent versions, or who approved what are hardest to reconstruct after the project has ended, which is exactly when they’re usually asked for.

STAPLE tries to make metadata collection cheap by attaching it to work you’re already doing, rather than treating it as a separate write-up at the end of a project.

4.3 Forms: How STAPLE Collects Metadata

In STAPLE, Forms are the structured templates used to collect metadata. A form defines *what* information should be collected (its schema) and *how* it is displayed to the person filling it out (its layout). Forms are built independently of any one project, saved to your account, and then reused across as many projects and tasks as you like.

STAPLE includes a library of Form Templates covering common needs (e.g., ethics approvals, dataset descriptions) so you have a starting point even if you’ve never designed a metadata schema before. You can also build a form entirely from scratch.

4.4 Building a Form

Forms are created and edited on the main **Forms** page (from the main dashboard navigation), not inside a specific project — this is what lets the same form be reused across projects.

- Go to the **Forms** page and click **Create New Form** (or **Add Form Templates** to start from an existing template instead of a blank form).

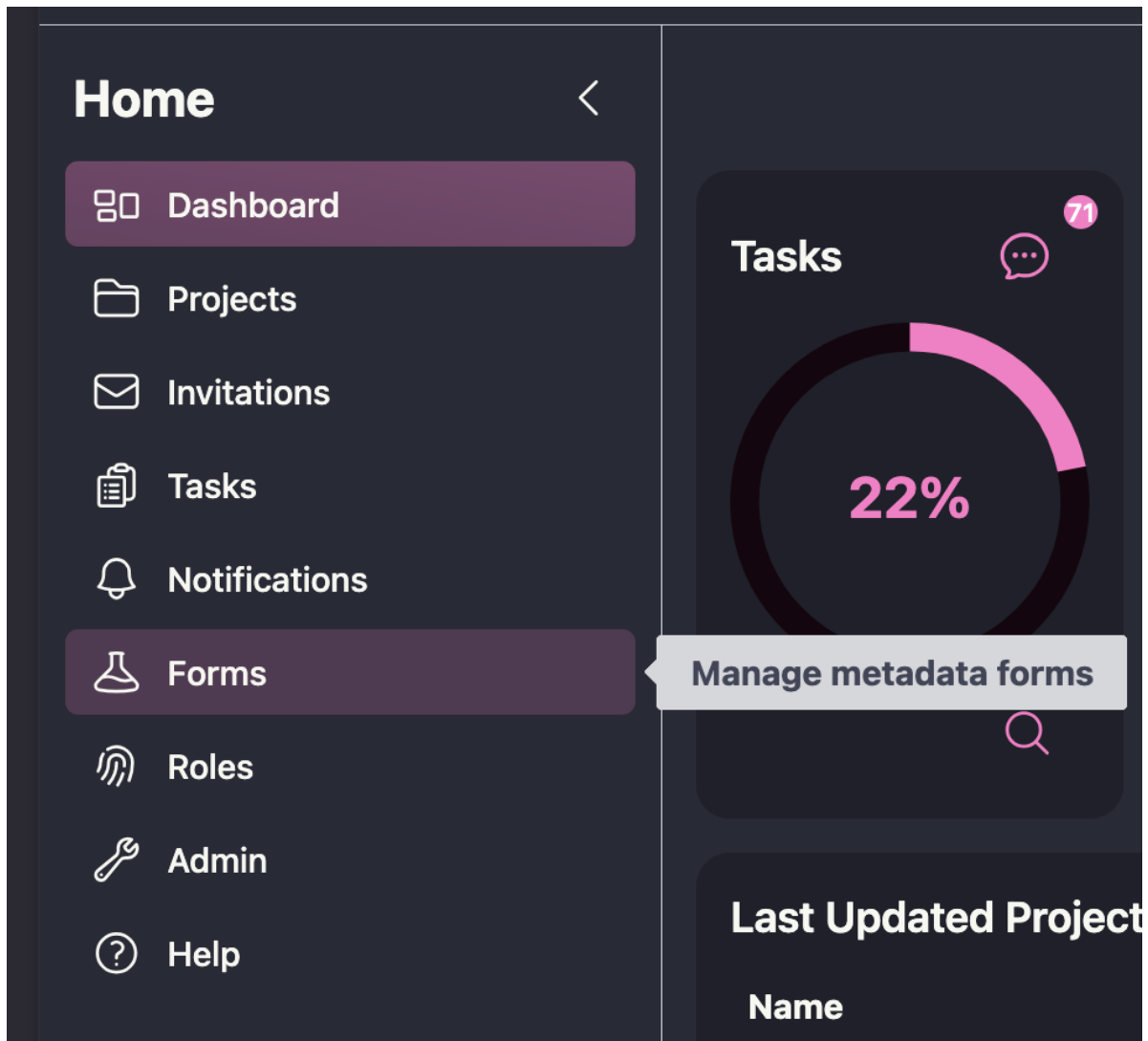


Figure 4.1: The Forms link in the main navigation sidebar.

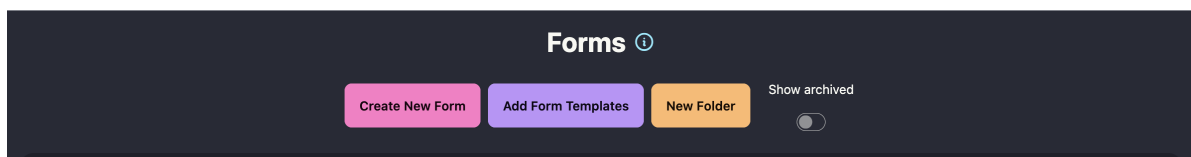


Figure 4.2: The Forms page, with buttons to create a new form, add form templates, or create a folder.

- This opens the form builder, which has three working views:

- **Visual Builder:** add fields (text, numbers, dates, choices, etc.) and configure their labels, descriptions, and requirements without writing JSON by hand.
- **JSON Builder:** edit the underlying JSON Schema and UI Schema directly, useful if you're importing an existing schema or want finer control.
- **Preview:** see the form exactly as a contributor filling it out would.

The screenshot shows the 'Visual Builder' tab of a form editor. On the left is a vertical sidebar with icons for navigation. The main area has a dark background. At the top, there are three tabs: 'Visual Builder' (active), 'JSON Builder', and 'Preview'. A 'Save Form' button is in the top right. Below the tabs, there are two main sections: 'Form Name' and 'Form Description'. The 'Form Name' section has a text input field with the placeholder 'Title'. The 'Form Description' section has a text input field with the placeholder 'Description'. Above the 'Form Description' input, there are three buttons: 'Edit' (active), 'Preview', and 'Supports Markdown'. Below the 'Form Description' input, there is a dashed box with a '+' sign, indicating where to add new fields.

Figure 4.3: The Visual Builder, showing the Form Name and Description fields to fill in before adding items.

- Before adding any fields, fill in the **Form Name** and **Form Description** at the top of the Visual Builder. The description supports Markdown and appears as help text at the top of the form, so contributors see it before they start filling anything in.

This screenshot shows the 'Visual Builder' interface with the 'Form Name' and 'Form Description' fields filled out. The 'Form Name' field contains the text 'Example Form'. The 'Form Description' field contains the text 'I am an example form!' followed by a line of italicized text '*Style with markdown!*' and a numbered list: '1) create a list' and '2) instructions to collaborators'. The 'Edit' button is still active, and the 'Supports Markdown' text is visible. The dashed box with the '+' sign is still present at the bottom.

Figure 4.4: Filling in the Form Name and Description on the Edit tab. Description supports Markdown formatting.

The image shows a form builder interface. At the top, there's a 'Form Name' field with the text 'Example Form'. Below that is a 'Form Description' section. It has three tabs: 'Edit', 'Preview' (which is active and highlighted in pink), and 'Supports Markdown'. The 'Preview' tab shows the rendered description: 'I am an example form!' followed by 'Style with markdown!' and a numbered list: '1. create a list' and '2. instructions to collaborators'.

Figure 4.5: The Preview tab showing the Form Description rendered from Markdown.

- Click the + below the description to add your first item. Choosing **Item** opens a new field card; choosing **Section** creates a labeled group you can add items into (useful for organizing a long form, e.g., separating “Ethics” fields from “Data” fields).

The image shows a 'Create New' popover. At the top, there's a dashed box with a '+' icon and the text 'Add a new item or section'. Below this is a 'Create New' section with two radio buttons: 'Item' (which is selected) and 'Section'. At the bottom of the popover are two buttons: 'Cancel' and 'Create'.

Figure 4.6: The Create New popover, letting you add an Item or a Section.

The image shows a form builder interface. At the top, there's a header 'New Input 1' with a right arrow icon. Below this is a dashed box with a '+' icon, indicating where to add a new item or section.

Figure 4.7: A newly added item, collapsed, with another Add button below it for the next item or section.

- Every item needs its own information filled in before it’s meaningful — the item is created blank, so you fill in:
 - **Variable Name:** the key used when the data is exported (e.g., in the downloaded JSON).
 - **Display Name:** the label shown on the form itself.
 - **Description:** help text shown under the field to the person filling it out.
 - **Item Type:** what kind of input this is (see [Item Types](#) below).
 - **Ontology ID** (optional): binds the field to a standard vocabulary code (e.g., NCIT:C25150) to improve findability if your field maps onto an existing standard.

▼ Display Name!

Variable Name ⓘ

downloadName

Display Name ⓘ

Display Name!

Description ⓘ

Edit Preview Supports Markdown

A description is small text below the display name.

Item Type ⓘ

Short Answer

Default Value

Default

Ontology ID (Optional) ⓘ

e.g. NCIT:C25150 ⓘ

Figure 4.8: An expanded item card, with Variable Name, Display Name, Description, Item Type, Default Value, and optional Ontology ID fields.

- You can also toggle **Required** at the bottom of the card, and use the pencil icon to open additional, type-specific settings (validation rules, default values, and dependencies — covered below). The trash icon deletes the item, and the arrows icon lets you drag it to reorder items or move them between sections.

Ontology ID (Optional) ⓘ

e.g. NCIT:C25150

☐ Required  

Figure 4.9: The Required toggle, and the pencil (additional settings) and trash (delete) icons at the bottom of an item card.

- Click **Save Form**. Once saved, the form gains an **Information** tab where you can:
 - Add **tags** and assign it to a **folder** to keep your form library organized.
 - View its Form Versions — every save creates a new version, and earlier versions stay available for tasks that already used them, so editing a form does not silently change data already collected under an older version.

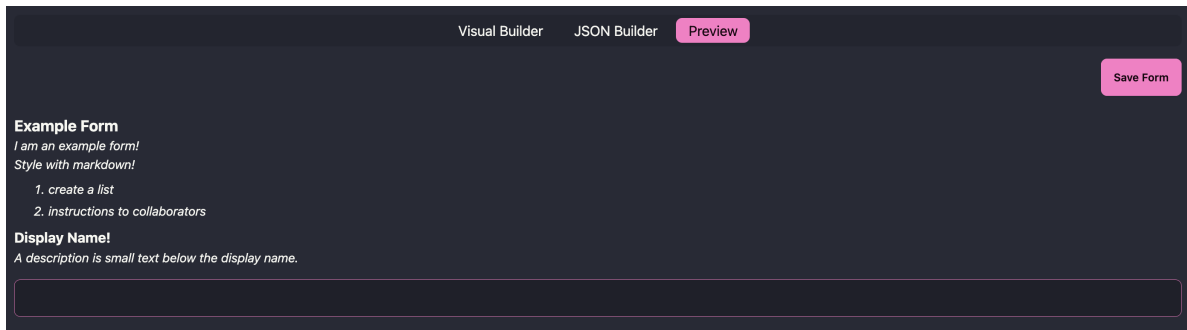


Figure 4.10: The Preview tab, showing the saved form as a contributor would see it.

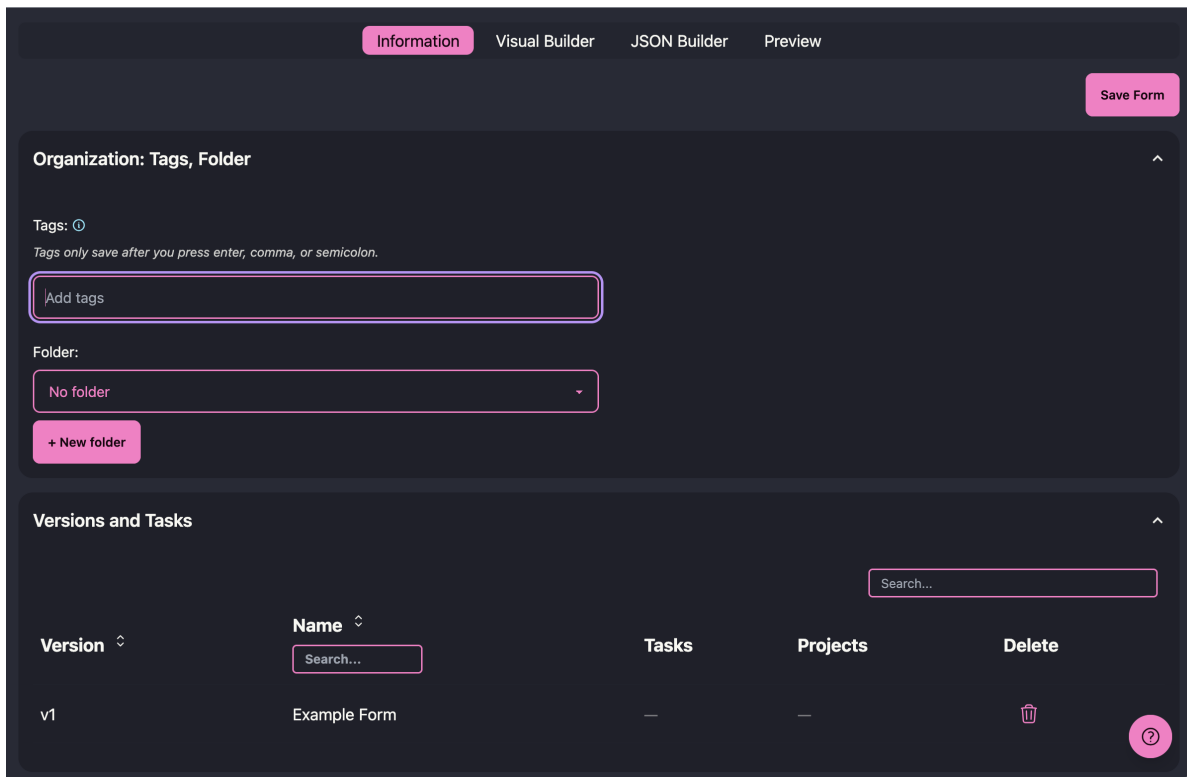


Figure 4.11: The Information tab, showing tags, folder, and the Versions and Tasks table for the saved form.

A form only needs to be built once. From here on, it's a reusable asset you attach to tasks whenever you need to collect that kind of metadata.

4.5 Item Types and Rules

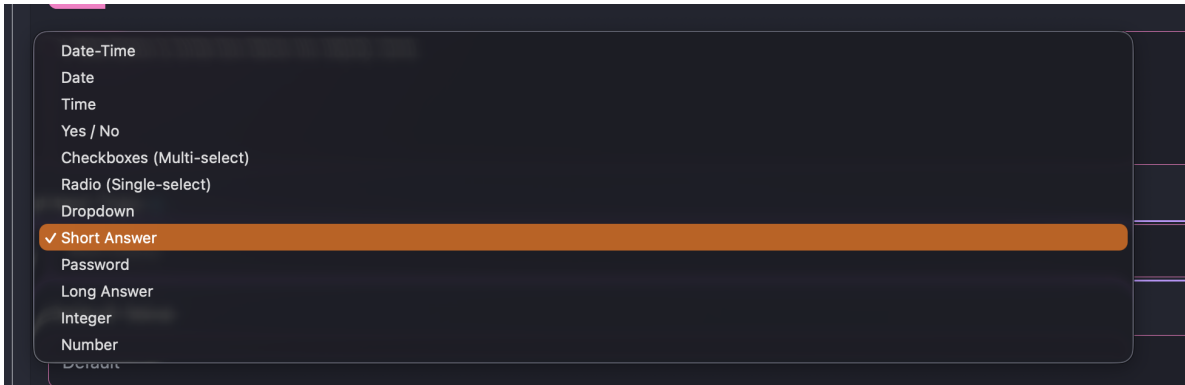


Figure 4.12: The Item Type dropdown, listing all available field types.

Each item you add to a form has an **Item Type**, which determines what kind of input the contributor sees and what the value looks like when the form data is exported. The Visual Builder currently supports:

- **Short Answer:** a single line of text. Its settings (via the pencil icon) let you set minimum/maximum length, a regular expression pattern the input must match, a common format (email, hostname, URI, or a custom regular expression), a browser autocomplete category, and a placeholder.
- **Password:** a masked short-answer field, useful for entering credentials that shouldn't be shown on screen.
- **Long Answer:** a multi-line text box, with the same length/pattern settings as Short Answer.
- **Integer** and **Number:** numeric input, with settings for a required multiple (e.g., must be a multiple of 5), and a minimum/maximum (each of which can be marked *exclusive*, i.e., the value must be strictly greater/less than the bound rather than equal to it).
- **Date**, **Time**, and **Date-Time:** date and/or time pickers.
- **Yes / No:** a checkbox for boolean (true/false) values.
- **Radio (Single-select)** and **Dropdown:** choose one value from a list you define. You can give each stored value a different display label, and force the list to store numbers instead of text.
- **Checkboxes (Multi-select):** choose any number of values from a list you define, stored as an array.
- **Reference:** reuse a field definition you've already created elsewhere in the same form, so you don't have to redefine the same structure twice.

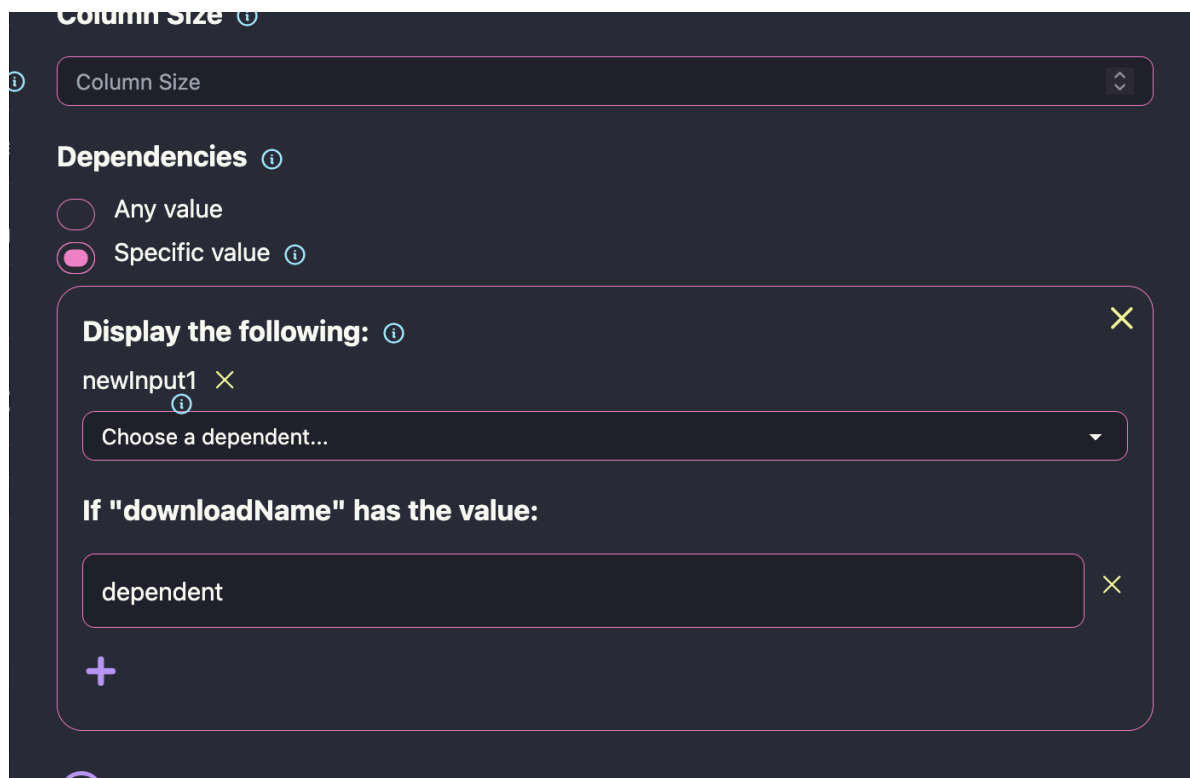
Every item can also be marked **Required**, which prevents the form from being submitted until that item has a value.

4.5.1 Dependencies

Any item can control whether *other* items on the form are shown, using the **Dependencies** setting (in the pencil-icon settings for that item). This lets you build forms that only ask follow-up questions when they're relevant, instead of showing every field to every contributor. There are two modes:

- **Any value:** the dependent item(s) appear as soon as the controlling item has *any* answer.
- **Specific value:** the dependent item(s) appear only when the controlling item is set to a particular value (or one of a set of values) that you choose.

You can add multiple dependency relations to the same item, and a dependent item shows a “Depends on ____” indicator in the builder so it's clear which items are conditional. For example, a Yes/No item “Did this study collect human data?” could be set so that answering “Yes” reveals a Short Answer item for “IRB/Ethics approval number,” which stays hidden for studies that don't need it.



The screenshot shows a dark-themed settings panel for a form item. At the top, there's a 'Column Size' dropdown. Below it is the 'Dependencies' section, which has two radio buttons: 'Any value' and 'Specific value'. The 'Specific value' option is selected. Under 'Specific value', there's a section titled 'Display the following:' which contains a list of dependent items. The first item is 'newInput1'. Below this, there's a dropdown menu labeled 'Choose a dependent...'. Further down, there's a section titled 'If "downloadName" has the value:' which contains a text input field with the word 'dependent' inside. At the bottom of the panel, there's a purple plus sign icon.

Figure 4.13: The Dependencies setting on an item, set to “Specific value”: this item's dependent field only displays when “downloadName” has a matching value.

4.6 Using a Form in a Task

Once a form exists, it can be required as part of completing a Task:

1. In a project, create a new task or edit an existing one.
2. Open the **Details: Instructions, Dates, Forms, Roles** section of the task form.
3. Click **Assign Form**. This opens a list of the available forms (with their version, creator, and creation date) — select the one the task should require, then confirm.
4. Save the task as usual. The assigned contributor or team now sees this form as part of the task.

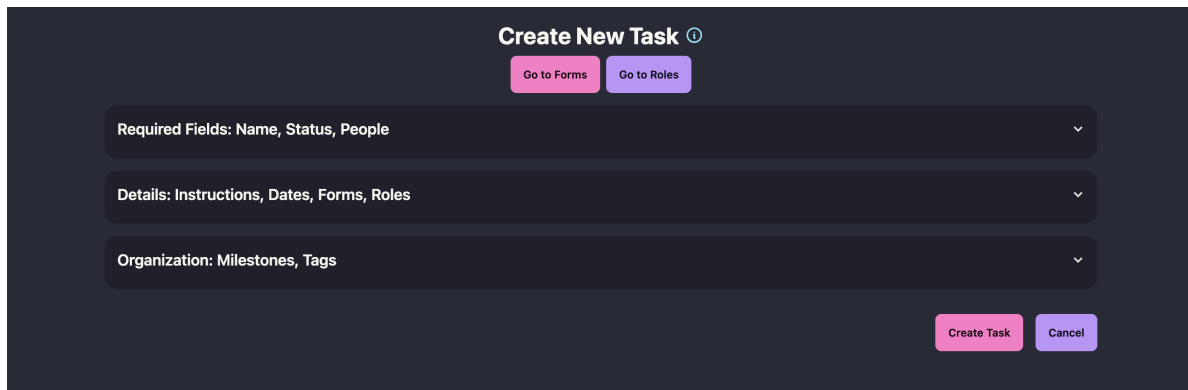


Figure 4.14: The Create New Task modal, with collapsible sections for required fields, details, and organization.

Details: Instructions, Dates, Forms, Roles

Task Instructions:

Edit

Preview

Supports *Markdown* formatting.

Add Instructions

Start Date:

mm / dd / yyyy

Deadline:

mm / dd / yyyy

Assign Form

Assign Role(s)

Figure 4.15: The Details section of a task, with Start Date, Deadline, Assign Form, and Assign Role(s) options.

Select Form

Search...

Select	Label ▾ Search...	Version ▾ Search...	Project Manager ▾ Search...	Created ▾ Search...
☐	Example Form	1	Erin Buchanan	8/31/2026
☐	Contributor Information (Assessment)	4	Erin Buchanan	8/14/2026
☐	Long Term Email	1	Erin Buchanan	12/10/2025
☐	Project Document	1	Erin Buchanan	12/3/2025
☐	Contributor Information (CREP)	2	Erin Buchanan	8/26/2025

<< < > >>

Page 1 of 3 | Go to: 1 ▾

Show 5

Save Close

Figure 4.16: The Select Form modal, listing available forms with their version, project manager, and creation date.

When a contributor works on the task, the assigned form appears as part of completing it. They fill in the requested fields and submit; STAPLE stores the submitted values as that task’s metadata and time-stamps the submission alongside the rest of the Task Logs history. If a mistake is made, the form can be reopened and resubmitted, and the previous entry is replaced.

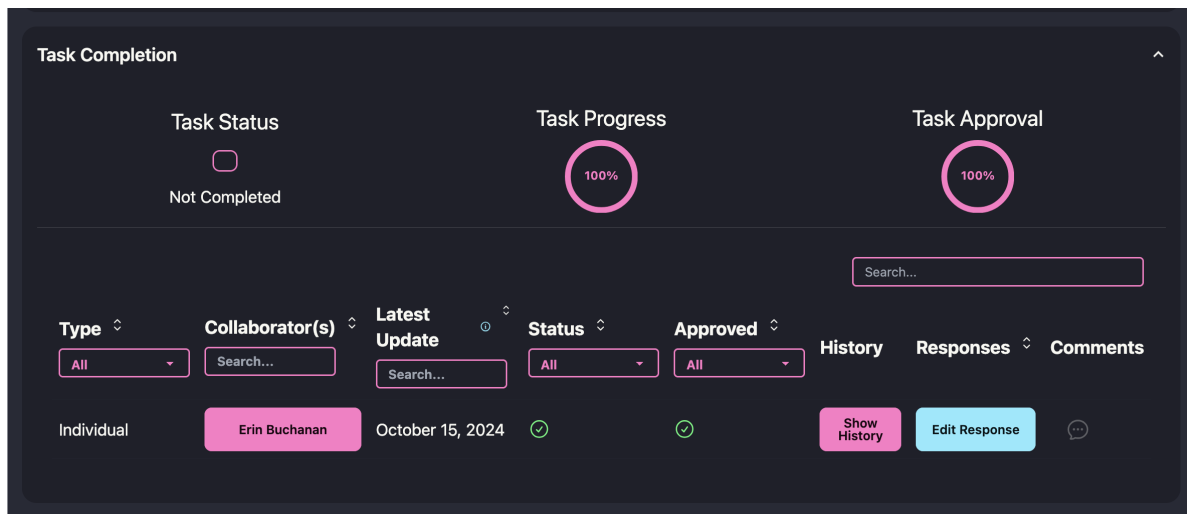


Figure 4.17: The Task Completion panel, showing task status, progress, and each contributor's response with options to show history or edit the response.

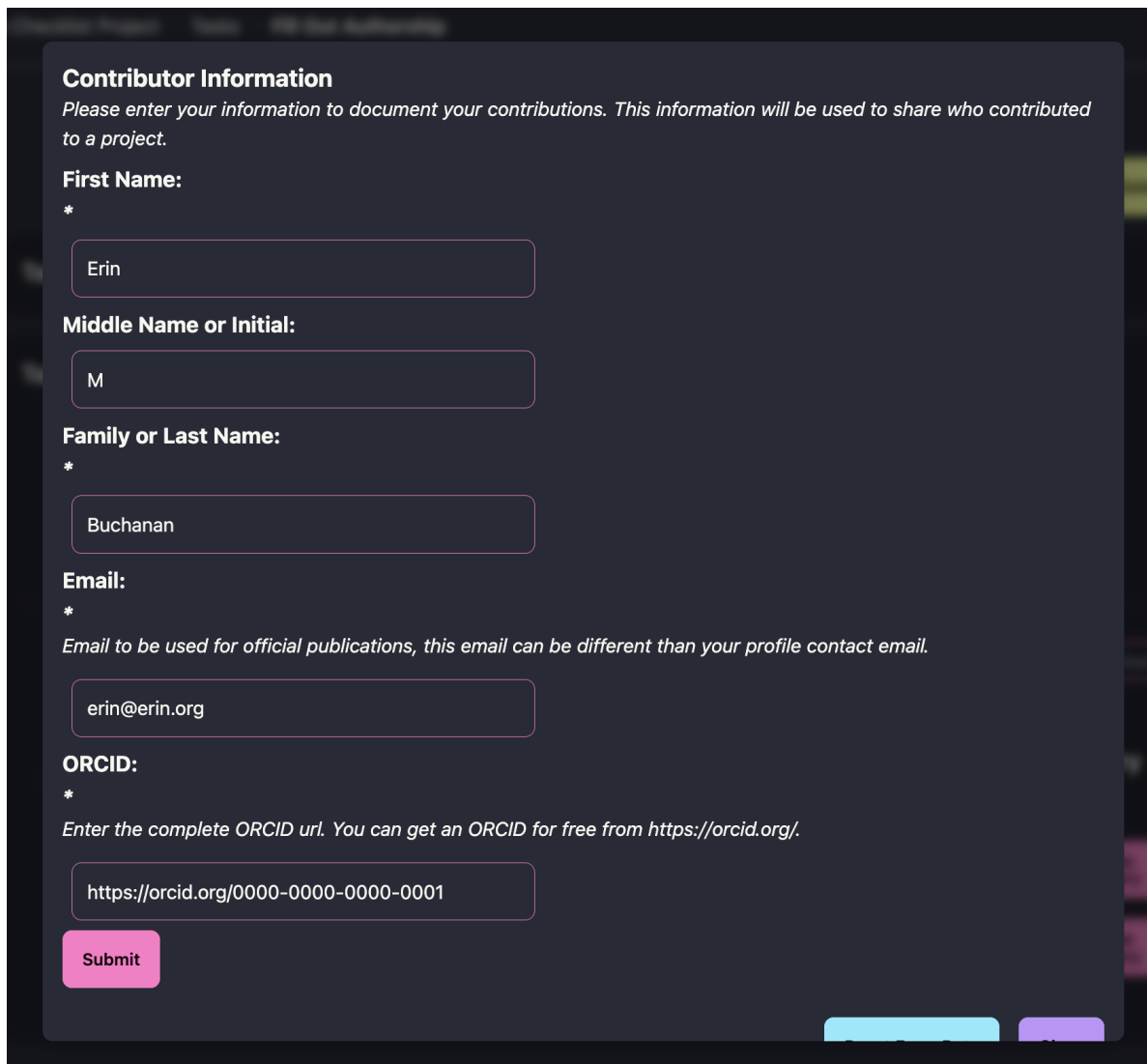
The image shows a web form titled "Contributor Information" with a dark background. The form contains several input fields and labels. At the top, the title "Contributor Information" is in bold. Below it is a paragraph: "Please enter your information to document your contributions. This information will be used to share who contributed to a project." The form has five main sections, each with a label and a required field indicator (*): 1. "First Name:" with an input field containing "Erin". 2. "Middle Name or Initial:" with an input field containing "M". 3. "Family or Last Name:" with an input field containing "Buchanan". 4. "Email:" with an input field containing "erin@erin.org". Below this field is a note: "Email to be used for official publications, this email can be different than your profile contact email." 5. "ORCID:" with an input field containing "https://orcid.org/0000-0000-0000-0001". Below this field is a note: "Enter the complete ORCID url. You can get an ORCID for free from https://orcid.org/." At the bottom left of the form is a pink "Submit" button. At the bottom right, there are two colored bars, one blue and one purple.

Figure 4.18: The assigned form as a contributor sees it while completing the task — here, a Contributor Information form.

Note: once a contributor has started completing a task, **both the task and its form are locked** — you can no longer change which form is assigned to that task, since that would alter the schema against which data has already been collected. If the form itself needs to change, edit it anyway: saving your changes doesn't overwrite the version already in use, it creates a new Form Version. The in-progress task keeps its original, locked version and whatever data has already been collected under it, while the new version becomes immediately available to assign to any task — including a new task if this one needs to move to the updated version.

4.7 Where Form Data Lives Afterward

Metadata collected through forms doesn't disappear into the task it was attached to. At the project level, the Form Data page gives a centralized, exportable view of every completed form across the project, and the **Summary** page rolls metadata, contributors, tasks, and outputs together into a shareable Project Summary — available as both a machine-readable JSON export and a human-readable report. This is what makes the metadata you collect along the way reusable later, by your collaborators, by journals and funders, or by your own future self.

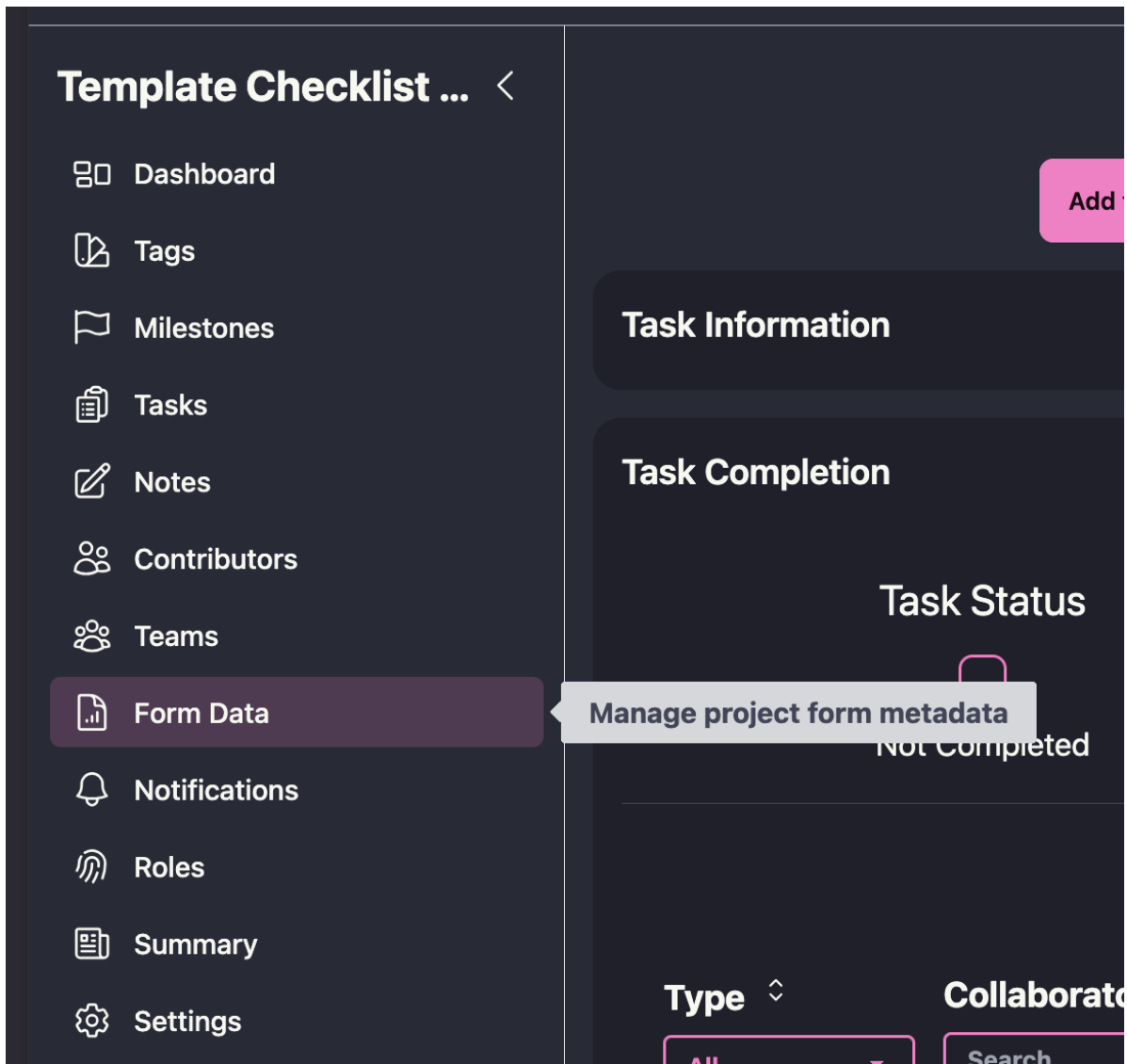


Figure 4.19: The Form Data link in a project's sidebar navigation.

Form Data ⓘ				
Search...				
Task ⌵	Required Form ⓘ ⌵	Completion ⌵	Approved ⌵	Responses
Search...	Search...	Min (100) ⌵ Max (100) ⌵	Min (0) ⌵ Max (100) ⌵	
Authorship Information	Contributor Information	100%	100%	
Fill Out Authorship	Contributor Information	100%	0%	

Figure 4.20: The Form Data page, listing each task’s required form alongside its completion and approval percentages, with a link to the responses.

Review Responses: <i>Fill Out Authorship</i> ⓘ						
Required Form Edit Responses Download JSON Download XLSX Download ZIP						
Search...						
Completed By ⌵	Changed At ⌵	Email: ⌵	First Name: ⌵	Family or Last Name: ⌵	ORCID: ⌵	Mid ⌵
Search...	Search...	Search...	Search...	Search...	Search...	Se
Erin Buchanan	October 18, 2024 at 14:11:02	erin@erin.org	Erin	Buchanan	https:// orcid.org/0000-0000-0000-0001	M
aggieerin	October 28, 2025 at 01:45:47	aggieerin@gmail.com	Erin	Buchanan	https:// orcid.org/0000-0000-0000-0001	M

Figure 4.21: The Review Responses page for a specific task, with options to download the collected data as JSON, XLSX, or a ZIP of attached files.

5 Installation Guide

STAPLE is available as a hosted service at <https://app.staple.science>, which is the simplest way to get started. You can also go to <https://app.staplescience.com> if your internet provider blocks `.science` addresses.

This guide is for individuals or organizations who would like to host their own instance of STAPLE. Running your own version gives you full control over your data, configuration, and integration with your existing infrastructure.

5.1 Assumptions

This guide assumes you meet the following prerequisites:

- You are working on a Linux server. We use Ubuntu 20.04 in this guide, but the steps should be consistent for most Linux distributions.
- You have shell/terminal access to the server.
- You have worked with the command line before.
- You have `sudo` privileges to install packages and manage services.
- You understand what hidden files are (and how to view them).
- You have a basic understanding of SQL and databases (familiarity with PostgreSQL is not required).
- You know what Apache and/or Nginx servers are and how to edit their configuration files.
- You can imagine yourself using git (even if you're new to it).

5.2 Local Development or Hosting

You can also run STAPLE locally on your own computer, with a few limitations.

- If you are using a Linux system, you can follow the same instructions described in this guide from Blitz/JS Installation through Starting the App – Production.
- On Windows, you can run STAPLE as well, but you'll need to use a shell environment (such as PowerShell, Git Bash, or WSL2) to follow the same command-line instructions.

- On macOS, the setup is nearly identical to Linux, and you can use the standard Terminal application to run the commands.

When running locally, you will be the only user able to enter data. The application is not hosted on the web, so others will not be able to connect.

Despite this limitation, you will still have access to the full functionality of project management and metadata entry/output. This makes local installation useful for:

- Testing the system before deploying to a server
- Exploring functionality on your own
- Managing personal projects without a shared server

5.3 Installation Steps

5.3.1 Web Server Installation

STAPLE requires a web server to handle requests. You can choose between **Apache** or **Nginx**:

- **Apache** – [Installation Instructions](#)
- **Nginx** – [Installation Instructions](#)

Important: Only install one. Apache and Nginx don't play nicely together if they're both running on the same server.

We recommend using **Nginx**, and our production server is configured with Nginx. This step is only required for people who want to host the software online. You do not need it for local development or use.

5.3.2 Blitz/JS Installation

STAPLE is built with **Blitz.js**, which runs on Node.js. To install the required tools:

- Install Node.js and npm
 - Follow the official installation instructions: [Node.js Downloads](#)
 - Required versions:
 - * **Node.js**: v18+
 - * **npm**: v9+
 - Verify installation with:

```
node -v
npm -v
```

- (Optional) Yarn: You may use Yarn if you prefer, but this guide assumes **npm**.
- Install Blitz.js

– Run:

```
npm install -g blitz
```

- Verify installation (version **v2+** required):

```
blitz -v
```

Note: Depending on your server setup, you may need to prepend commands with `sudo`.

5.3.3 Clone the Repository

Next, you'll need to copy the STAPLE source code onto your server.

- Install Git (if not already installed):

```
sudo apt-get update
sudo apt-get install git
```

- Navigate to your web directory (commonly `/var/www/html/` or `/var/www/`):
- *Note:* you can use any folder on your computer if you are not hosting the software online or want to do local development.

```
cd /var/www/html/
```

- Clone the repository:
 - Follow the [GitHub guide](#) on cloning repositories.
 - * Or simply run:

```
git clone https://github.com/STAPLE-verse/STAPLE.git
```

- This will create a new folder named STAPLE containing all the files you need.
- Move into the project folder:

```
cd STAPLE
```

5.3.4 Database Installation

STAPLE requires a PostgreSQL database.

- Install PostgreSQL
 - Ensure that you have a local PostgreSQL service running.
 - Follow this guide for installation: [Postgres Setup Instructions](#).
 - During installation:
 - * Write down the superuser credentials (important on non-Linux systems).
 - * Other databases may work, but you would need to modify the STAPLE codebase to support them.
- Create Databases
 - Open a terminal and run:

```
# Switch into PostgreSQL as the postgres superuser
sudo -u postgres psql
```

```
# Create databases for STAPLE
CREATE DATABASE staple;
CREATE DATABASE staple_test;
```

- Create a User
 - It's best not to use the default postgres user for STAPLE. Instead, create a new user:
 - Replace username and password with your desired values (keep the quotes).

```
CREATE USER username WITH PASSWORD 'password';
ALTER ROLE username CREATEDB;
```

- Check Databases
 - Inside the PostgreSQL prompt, you can verify:

```
\l -- lists databases
\c staple -- connect to the STAPLE database
\dn -- lists schemas
```

- You should see something like the following using those commands:

```
postgres=# \c staple
You are now connected to database "staple" as user "postgres".
staple=#
```

```
staple=# \dn
List of schemas
  Name  | Owner
-----+-----
 public | staple
(1 row)
```

```
staple=#
```

```
postgres=# \l
```

List of databases						
Name	Owner	Encoding	Collate	Ctype	Access privileges	
postgres	postgres	UTF8	en_US.UTF-8	en_US.UTF-8		
staple	postgres	UTF8	en_US.UTF-8	en_US.UTF-8	=Tc/postgres	+
					postgres=CTc/postgres	+
					staple=CTc/postgres	+
					staple_admin=CTc/postgres	
staple_test	postgres	UTF8	en_US.UTF-8	en_US.UTF-8		
template0	postgres	UTF8	en_US.UTF-8	en_US.UTF-8	=c/postgres	+
					postgres=CTc/postgres	
template1	postgres	UTF8	en_US.UTF-8	en_US.UTF-8	=c/postgres	+
					postgres=CTc/postgres	

```
(5 rows)
```

- Grant Privileges
 - Finally, grant permissions for your new user to manage the staple schema:

```
GRANT ALL ON SCHEMA public TO username;
```

- Exit PostgreSQL using \q

5.3.5 Connect Database to STAPLE App

- Now we'll connect the PostgreSQL database you created to the STAPLE application.
- Navigate to your STAPLE project folder

```
cd /var/www/html/STAPLE
ls -al
```

- You should see a file named `.env` along with other project files.
- You may need to enable hidden files to see `.env` files.
- Example:

```
erin@staple:/var/www/html/STAPLE$ ls -al
total 764
drwxr-xr-x 13 root root 4096 Oct 24 06:17 .
drwxr-xr-x  6 root root 4096 Oct 24 05:02 ..
drwxr-xr-x  3 root root 4096 Oct 24 04:27 db
-rw-r--r--  1 root root  175 Oct 24 04:27 .editorconfig
-rw-r--r--  1 root root  494 Oct 24 06:17 .env
```

- Create a local environment file
 - Copy the `.env` file and rename it to `.env.local`:

```
cp .env .env.local
```

– Open it for editing:

```
nano .env.local
```

- Add required environment variables
 - At minimum, your `.env.local` file needs these:

```
# This env file should NOT be checked into source control
# Use this file for local overrides

# Replace username:password with your Postgres user and password
DATABASE_URL=postgresql://username:password@localhost:5432/staple

# Generate a random session key
SESSION_SECRET_KEY=your_generated_session_key

# Where the app will live
APP_ORIGIN="http://localhost:3000/"
```

- To generate a `SESSION_SECRET_KEY`:

- Copy the output and paste it in place of your `_generated_session_key`.

```
openssl rand -hex 16
```

- Set up a test environment
 - Copy `.env.test` to `.env.test.local`:

```
cp .env.test .env.test.local
```

- Edit it to point to your test database:

```
DATABASE_URL=postgresql://username:password@localhost:5432/staple_test
```

5.3.6 Configure Email

STAPLE requires an email provider to send system emails (e.g., password resets, project invitations). You can use other options here, but we've configured three of them in our current code.

5.3.6.1 Secret Keys

- You must configure at least one of the following options:
- Gmail (App Passwords)
 - Create a Gmail App Password: [Google Guide](#)
 - Add to `.env.local`:

```
EMAIL_PASS="your_app_password"
```

- Amazon SES
 - Set up Amazon Simple Email Service (SES): [Amazon SES Docs](#)
 - Add to `.env.local`:

```
ACCESS_KEY="amazonACCESS"
SECRET_ACCESS_KEY="amazonSECRET"
```

- Resend
 - Get an API key from Resend.
 - Add to `.env.local`:

```
RESEND_API_KEY="resendAPI"
```

5.3.6.2 Edit the Mailer Configuration

In addition to adding one of the above environment variables, you'll need to edit the mailer configuration file so STAPLE knows which provider to use.

- Open the mailer configuration file:

```
nano integrations/mailer.js
```

- Delete non-used mailers: there are three chunks of code in this file - one for each type of email. You should delete the types you are not using to ensure the app runs. You can also insert your own mail options in this section.
- Edit the forgot password mailer:

```
nano mailers/forgotPasswordMailer.ts
```

- Change these sections to the mailer you want to use:

```
// import { Mailer } from "integrations/mailer"
import { createForgotPasswordMsg } from "integrations/emails"
// import { Amazon } from "integrations/mailer"
import { ResendMsg } from "integrations/mailer"

//send the email
await ResendMsg(createForgotPasswordMsg(to, resetUrl))
// await Amazon(createForgotPasswordMsg(to, resetUrl)) # or amazon
// await Mailer(createForgotPasswordMsg(to, resetUrl)) # or gmail
```

- Edit the emails.tsx file if you want to customize the emails that are sent with your own email address and logos.

```
nano integrations/emails.tsx
```

5.3.7 Install STAPLE Requirements

- Make sure you are in the STAPLE main folder before running these commands.
- Use the following (not the # line, these are notes) to install packages, tailwind, and daisyui.

```
# Install all project dependencies
npm install
```

```
# Install Tailwind CSS
blitz install tailwind
```

```
# Install DaisyUI
npm i -D daisyui@latest
```

- Set up the database schema:

```
# Generate Prisma client
blitz prisma generate
```

```
# Run migrations to create database structure
blitz prisma migrate dev
```

5.3.8 Setting Up the STAPLE Viewer Worker

The Viewer Worker is a background process that supports the Build Summary Viewer in STAPLE. It listens for Redis messages and processes summary-building tasks asynchronously.

- Run the Viewer Worker (manual test)
- Before setting it up as a service, you can test the worker manually to confirm it runs correctly:

```
sudo npm run run-viewer-worker
```

- This command should start the background process and display console logs from the viewer worker.
- Install and enable Redis: The viewer worker relies on Redis for managing background jobs and caching.

```
sudo apt update
sudo apt install redis-server
sudo systemctl enable redis-server
sudo systemctl start redis-server
```

- To confirm Redis is running:

```
sudo systemctl status redis-server
```

- Create the Viewer Worker service: next, create a systemd service so the viewer worker runs automatically on startup.
- Create a new service file:

```
sudo nano /etc/systemd/system/viewer-worker.service
```

- Add the following configuration (update the Node.js path if necessary):

```
[Unit]
```

```
Description=STAPLE Viewer Worker
```

```
After=network.target
```

```
[Service]
```

```
WorkingDirectory=/var/www/html/STAPLE
```

```
ExecStart=/home/ubuntu/.nvm/versions/node/v20.17.0/bin/node /var/www/html/STAPLE/src/summary.
```

```
Restart=on-failure
```

```
Environment=NODE_ENV=production
```

```
[Install]
```

```
WantedBy=multi-user.target
```

Note: If your Node.js binary is installed globally (for example at /usr/bin/node), you can replace the ExecStart line with:

```
ExecStart=/usr/bin/node /var/www/html/STAPLE/src/summary/utils/viewerWorker.js
```

- Enable and start the service

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable viewer-worker
```

```
sudo systemctl start viewer-worker
```

- Check the service status:

```
sudo systemctl status viewer-worker
```

- Viewing logs: To monitor the viewer worker's activity:

```
sudo journalctl -u viewer-worker -f
```

- This will stream logs from the worker in real time, which is useful for debugging summary build issues.
- Restarting the service: After making changes to STAPLE's summary logic or viewer-Worker.js, restart the service:

```
sudo systemctl restart viewer-worker
```

5.3.9 Starting the App - Local Testing

- Open a terminal window and navigate to the STAPLE folder you cloned:

```
cd /var/www/html/STAPLE
```

- Start the development server:

```
blitz dev
```

- Once the server is running, open your browser and go to: <http://localhost:3000> (or whatever address is shown in your terminal output).

5.3.10 Starting the App - Production

- Open a terminal window and navigate to the STAPLE folder you cloned:

```
cd /var/www/html/STAPLE
```

- Build the application:

```
blitz build
```

Note: If the build process produces errors, you'll need to fix these before you can continue. Common issues include missing dependencies or mismatched Prisma client versions. Check the terminal output for details. We do not update our dev or main branches without fixing these, but it may be that your edits for your server caused hiccups.

- Once the build completes successfully, start the app in production mode:

```
blitz start
```

5.3.11 Keeping the App Going

When running in production, you'll want STAPLE to automatically restart after crashes or reboots. The recommended way to do this on Linux is by creating a systemd service.

- Create a service file
 - Navigate to the systemd directory and create a new file (we'll call it `blitz.service`):

```
cd /etc/systemd/system/  
sudo nano blitz.service
```

- Example service file:
 - Replace `WorkingDirectory` with the path to your STAPLE installation.
 - You can also set specific users or production modes for security.

```
[Unit]  
Description=Starts the Blitz service.  
After=network.target
```

```
[Service]  
Type=simple  
WorkingDirectory=/var/www/html/STAPLE  
ExecStart=/usr/local/bin/blitz start  
Restart=always
```

```
[Install]  
WantedBy=default.target
```

- Enable and control the service
 - Once your file is saved, you can manage it with:

```
# Start the service  
sudo systemctl start blitz
```

```
# Check status (press "q" to quit)  
sudo systemctl status blitz
```

```
# Restart the service  
sudo systemctl restart blitz
```

```
# Stop the service
```

```
sudo systemctl stop blitz

# Enable service on boot
sudo systemctl enable blitz

# Disable service
sudo systemctl disable blitz

# Reset if failed
sudo systemctl reset-failed blitz
```

- Tutorial reference
 - If you're new to systemd services, see this helpful guide: [Service Installation Instructions](#)

5.3.12 Setting Up the Proxy (Nginx)

Now that STAPLE is running as a background service, you'll want to expose it through your web domain with Nginx.

- Navigate to your Nginx configuration folder

```
cd /etc/nginx/sites-enabled
```

- Create a new site config file
 - Name it after your site, e.g.:

```
sudo nano app.staple.science
```

- Add the server configuration
 - Replace app.staple.science with your domain and update the log path if needed:

```
server {
    listen 80;
    server_name app.staple.science;

    error_log /var/www/html/YOURFOLDER/logs/web-server.log;

    location / {
        proxy_pass http://localhost:3000/;
    }
}
```

```

        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}

```

Important: Make sure the `server_name` matches your domain name, and the `proxy_pass` points to the port where Blitz is running (localhost:3000 by default).

- Test your config:

```
sudo nginx -t
```

- If you see syntax is ok and test is successful, reload Nginx:

```
sudo systemctl reload nginx
```

5.3.13 Enable HTTPS

- Use Let's Encrypt to add SSL by changing the location of your app in the following:

```

sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d app.staple.science

```

- This will automatically configure HTTPS and renew your certificate.
- See [Installation Instructions](#).

5.3.14 Install pgAdmin (Optional)

pgAdmin provides a graphical interface for managing your PostgreSQL database. Follow these steps to install and configure pgAdmin on your server.

- Update your system:

```
sudo apt update
```

- Add the pgAdmin repository: Import the public key and add the pgAdmin APT repository:

```

curl -fsS https://www.pgadmin.org/static/packages_pgadmin_org.pub | sudo gpg --
dearmor -o /usr/share/keyrings/packages-pgadmin-org.gpg

```

```
sudo sh -c 'echo "deb [signed-by=/usr/share/keyrings/packages-pgadmin-org.gpg] https://ftp.pcs) pgadmin4 main" > /etc/apt/sources.list.d/pgadmin4.list && apt update'
```

- Install pgAdmin:

```
sudo apt install pgadmin4
```

- Run the setup webscript:

```
sudo /usr/pgadmin4/bin/setup-web.sh
```

- When prompted, enter the following credentials:
 - Email: your email
 - Password: your password
 - Note: these do not have to match any other accounts or the database.
- Adjusting Ports (if using Nginx): If your server is already running Nginx (which typically listens on port 80), you'll need to change Apache's default port to avoid conflicts.
- Edit Apache's port configuration:

```
sudo nano /etc/apache2/ports.conf
```

- Change Listen 80 to something else:

```
Listen 80
```

```
Listen 81
```

- Update the default virtual host:

```
sudo nano /etc/apache2/sites-available/000-default.conf
```

- Change this port to the one you picked earlier:

```
<VirtualHost *:80>
```

```
<VirtualHost *:81>
```

- Restart Apache:

```
sudo systemctl restart apache2
```

- Verify the port is open:

```
sudo ss -tulnp | grep :81
```

5.3.14.1 Accessing pgAdmin

- Once Apache is running, open pgAdmin in your browser:

`http://192.168.50.25:81/pgadmin4`

- Log in using the email and password you configured earlier.

5.3.14.2 Verifying PostgreSQL Connectivity

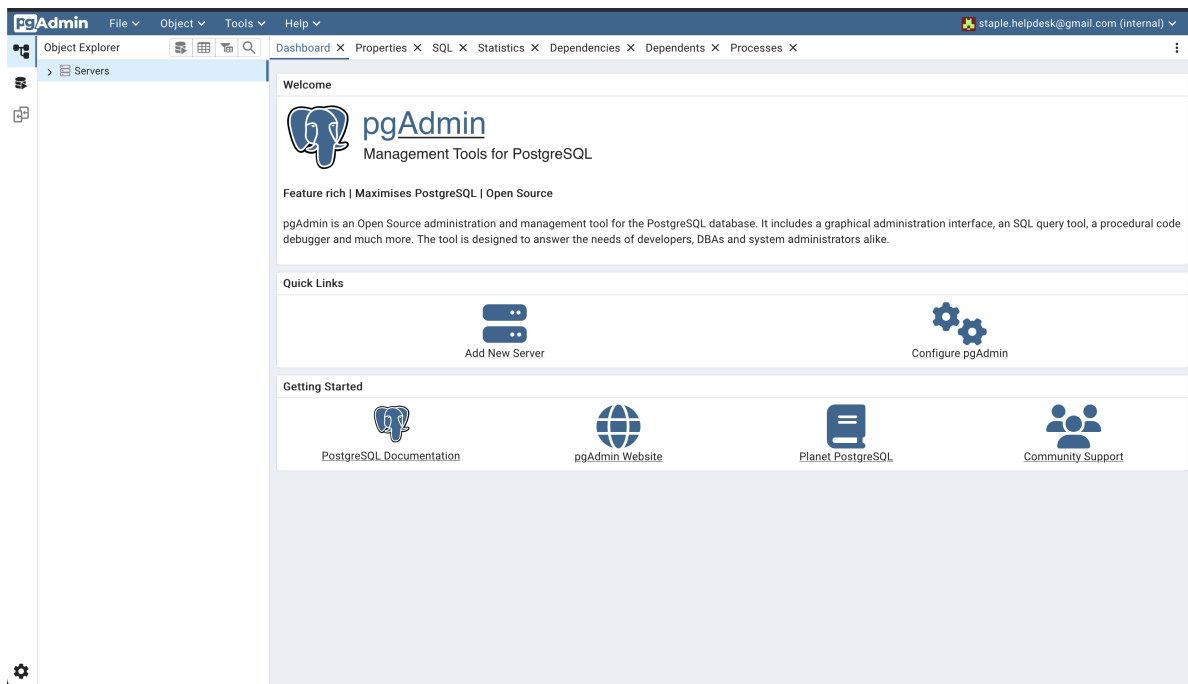
- Ensure that PostgreSQL is configured to accept connections:

```
sudo -u postgres psql -c "SHOW listen_addresses;"
```

The output should show:

*

- If not, update your PostgreSQL configuration (`/etc/postgresql/<version>/main/postgresql.conf`) and restart the PostgreSQL service.



5.3.14.3 Connecting to Your Database

- In pgAdmin, click Add New Server.

The screenshot shows the 'Register - Server' dialog box in pgAdmin. The 'General' tab is selected. The 'Name' field contains the text 'STAPLE'. The 'Server group' dropdown menu is set to 'Servers'. The 'Background' and 'Foreground' checkboxes are both unchecked. The 'Connect now?' toggle switch is turned on. The 'Shared?' toggle switch is turned off. The 'Shared Username' field is empty. The 'Comments' field is empty. At the bottom of the dialog, there are three buttons: 'Close', 'Reset', and 'Save'.

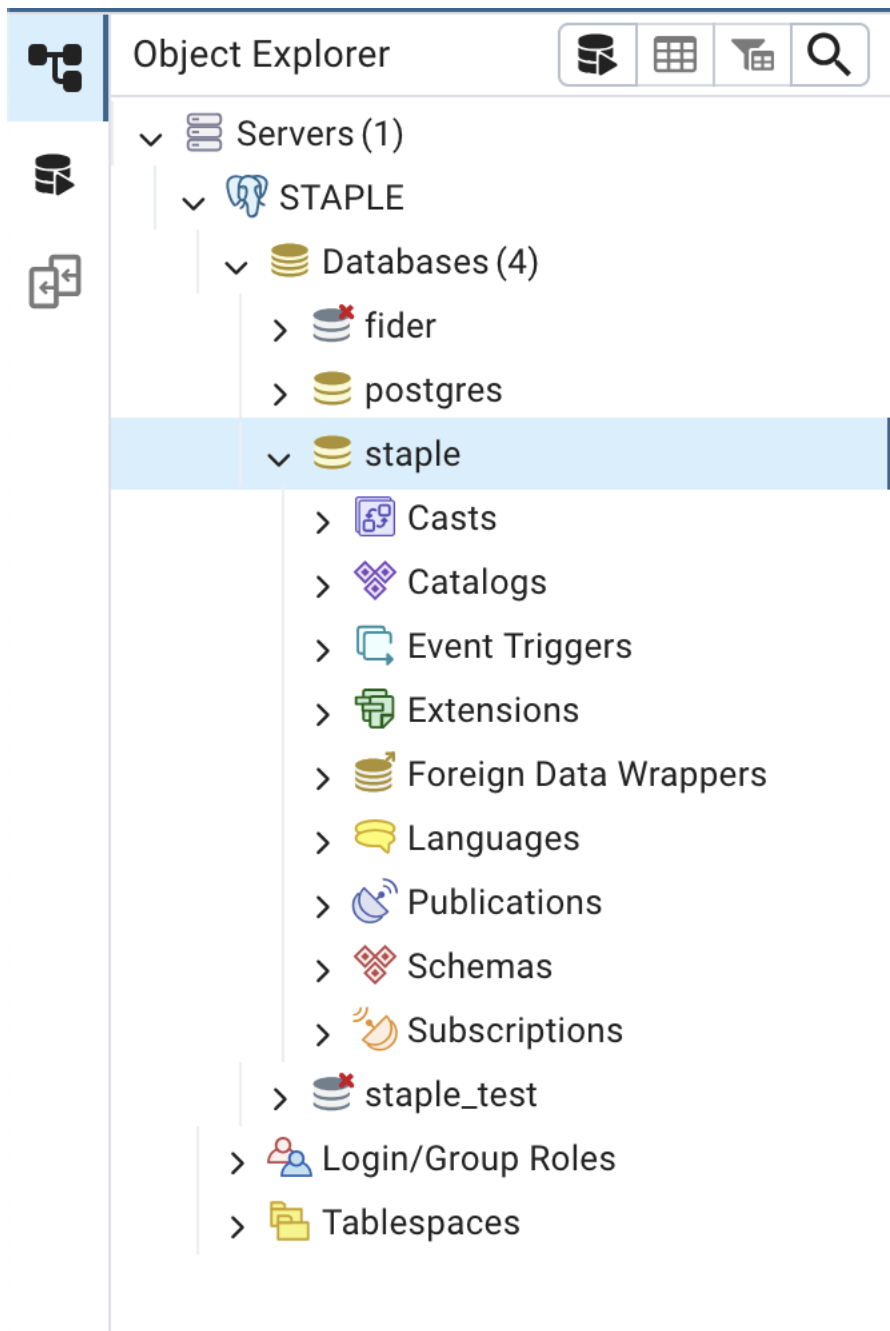
- Under the General tab, enter a name (e.g., STAPLE).

The screenshot shows a 'Register - Server' dialog box with the 'Connection' tab selected. The dialog has a title bar with a close button (X) and a menu icon. Below the title bar are tabs: 'General', 'Connection' (active), 'Parameters', 'SSH Tunnel', 'Advanced', and 'Tags'. The 'Connection' tab contains the following fields and controls:

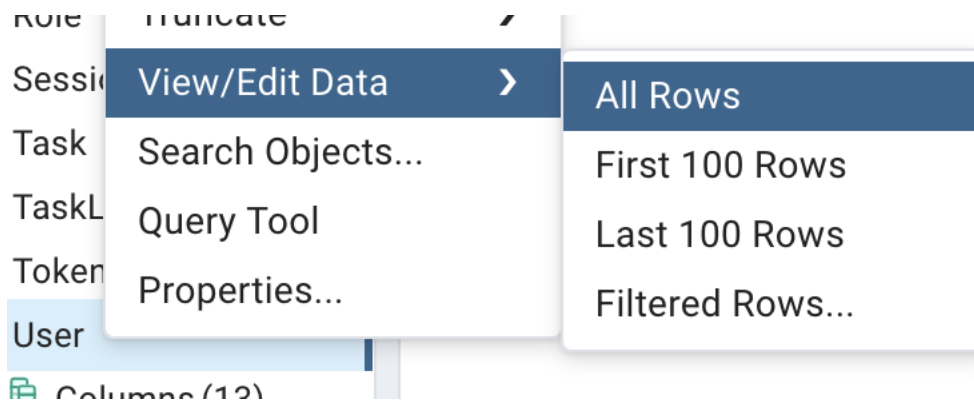
- Host name/address: localhost
- Port: 5432
- Maintenance database: postgres
- Username: staple
- Kerberos authentication?: ☐
- Password: (empty text field)
- Save password?: ☒
- Role: (empty text field)
- Service: (empty text field)

At the bottom of the dialog are three buttons: 'Close' (with an X icon), 'Reset' (with a circular arrow icon), and 'Save' (with a floppy disk icon). There are also two small icons (an 'i' and a '?') on the left side of the bottom bar.

- Under the **Connection** tab, provide:
 - Host name/address: localhost
 - Username: staple username set up earlier
 - Password: password set up earlier
 - Click Save.
- Once connected, you can navigate to:



- **STAPLE** → **Schemas** → **Public** → **Tables** → **User**
- Right-click on the **User** table to view or query data.



5.3.15 Updating STAPLE

To keep your STAPLE installation up to date with the latest features and fixes, follow these steps:

- Navigate to your STAPLE directory: Open a terminal and change into the folder where you originally cloned the STAPLE repository:

```
cd /path/to/staple
```

- Pull the latest changes: Update your local copy of the repository by pulling the newest code from the main branch:

```
git pull origin main
```

- Reinstall dependencies: Ensure all required Node modules are up to date:

```
npm install
```

- Run database migrations: Apply any new schema changes to your database:

```
blitz prisma migrate dev
```

- Rebuild the project: Rebuild STAPLE to compile updated files:

```
blitz build
```

- Restart the STAPLE service: Finally, restart the Blitz service so the new version takes effect:

```
sudo systemctl restart blitz.service
```

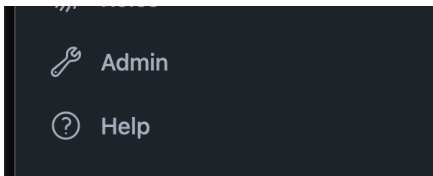
Tip: You can check that STAPLE is running correctly after the update by visiting your instance's URL or reviewing the logs:

```
sudo journalctl -u blitz.service -f
```

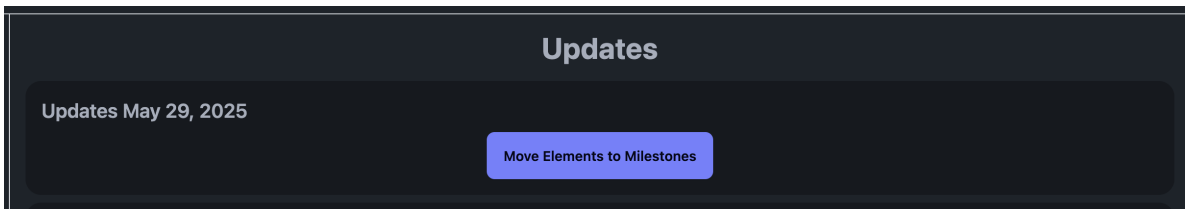
5.3.15.1 Database Migration

Every so often, we update features that require a manual migration of the database - for example, when we created Milestones, we had to copy over the data from the old naming system into the new. These types of changes are not handled by `blitz prisma migrate dev`.

- You can create a 'super user' by having a person with their `user.role == ADMIN` rather than `user`. This role adds a link on the main home side bar:



- You will then be able to run migrations within the app:



- We always add these buttons when applying this type of change to ensure it will not break the online database.

5.4 Common Issues & Fixes

5.4.1 Prisma / Database Errors

- Error: Error: P1000: Authentication failed against database server
 - Fix: Double-check your `DATABASE_URL` in `.env.local`. Make sure the username/password match the Postgres user you created.

- Error: Error: P3014: The underlying table does not exist
 - Fix: Run migrations again:

```
blitz prisma migrate dev
```

- Permission denied when running migrations
 - Fix: Ensure the Postgres user has privileges:

```
ALTER ROLE your_username CREATEDB;
GRANT ALL ON SCHEMA public TO your_username;
```

5.4.2 Build Errors

- Error: blitz build fails with missing dependencies
 - Fix: Run npm install again to ensure all dependencies are installed.
- Prisma client mismatch
 - Fix: Run:

```
blitz prisma generate
```

5.4.3 Email Issues

- No emails are being sent
 - Fix: Check that you kept only one mailer provider in mailer.ts.
 - Ensure the correct environment variables (EMAIL_PASS, ACCESS_KEY, SECRET_ACCESS_KEY, or RESEND_API_KEY) are set in .env.local.
 - For Amazon SES, make sure your MAIL_FROM address is verified.
 - For Gmail, use an App Password, not your regular password.

5.4.4 Nginx / Proxy Issues

- Nginx won't start
 - Fix: Run `sudo nginx -t` to test configuration. Correct any syntax errors before restarting.
- Site loads, but app doesn't respond

- Fix: Make sure blitz is running (check with `sudo systemctl status blitz`).
- Check that the `proxy_pass` in your Nginx config matches the Blitz port (localhost:3000 by default).
- SSL certificate not working
 - Fix: Ensure port 80/443 are open on your firewall.
 - Re-run certbot:

```
sudo certbot --nginx -d yourdomain.com
```

5.4.5 Service Issues

- Blitz doesn't restart after reboot
 - Fix: Make sure the service is enabled:

```
sudo systemctl enable blitz
```

- Service crashes with permission errors
 - Fix: Ensure the User in `blitz.service` has read/write access to the STAPLE folder.

5.4.6 General Tips

- After editing `.env.local`, `.env.test.local`, or `blitz.service`, restart the app:

```
sudo systemctl restart blitz
```

- Check logs:

```
journalctl -u blitz -f
```

Get help: contact [staple.helpdesk at gmail.com](mailto:staple.helpdesk@gmail.com).

6 Glossary

term	definition
authorship	Being listed as an author on a research paper, usually to recognize someone's contribution
Contributor/Member	An individual who actively participates in a project. Contributors may take on various roles within a project
contributorship	An alternative to traditional authorship that provides a more transparent account of individual contributions
CRedit	A standardized system for describing the specific roles individuals play in research
FAIR	FAIR stands for Findable, Accessible, Interoperable, and Reusable, a set of guidelines for data management
Form Data	The metadata that has been submitted through a project's forms. Form Data is stored in the project's metadata
Form Template	A pre-built form provided by STAPLE as a starting point for common metadata collection
Form Version	A snapshot of a form's schema created each time it is saved. Earlier versions remain available for reference
Forms	Structured templates for capturing important project metadata, such as study design, objectives, and outcomes
metadata	Metadata is information that describes other data. It helps explain what a dataset is, how it was collected, and how to use it
Milestones	Markers of significant progress within a project, such as completing data collection or analysis
Project	A container for a scientific effort, capturing the overall scope, goals, timeline, and associated resources
Project Summary	An automatically generated overview that consolidates key project information, including goals, progress, and outcomes
Roles	Defined levels of access and responsibility within a project (e.g., project manager, contributor, viewer)
Tags	Flexible labels used to categorize and filter tasks or projects. Tags help organize and search for specific information
Task	A discrete unit of work within a project, such as designing a survey, analyzing data, or writing a paper
Task Logs	A chronological record of all activity on a task, including assignments, updates, and completion status
Team	The group of contributors working together on a project. Teams may span laboratories, institutions, or countries
Transparency and Openness	A set of community-developed standards created to improve the transparency, reproducibility, and accessibility of research
User	Any person with access to STAPLE. A user may belong to multiple projects and have different roles within each

7 References

- Adan, C. (2023). The importance of diversity in clinical research. *British Journal of Nursing (Mark Allen Publishing)*, 32(18), 898–901. <https://doi.org/10.12968/bjon.2023.32.18.898>
- Allen, L., O’Connell, A., & Kiermer, V. (2019). How can we ensure visibility and diversity in research contributions? How the Contributor Role Taxonomy (CRediT) is helping the shift from authorship to contributorship. *Learned Publishing*, 32(1), 71–74. <https://doi.org/10.1002/leap.1210>
- Allen, L., Scott, J., Brand, A., Hlava, M., & Altman, M. (2014). Publishing: Credit where credit is due. *Nature*, 508(7496), 312–313. <https://doi.org/10.1038/508312a>
- Borghi, J. A., & Gulick, A. E. V. (2021). Data management and sharing: Practices and perceptions of psychology researchers. *PLOS ONE*, 16(5), e0252047. <https://doi.org/10.1371/journal.pone.0252047>
- Cummings, J. N., & Kiesler, S. (2005). Collaborative Research Across Disciplinary and Organizational Boundaries. *Social Studies of Science*, 35(5), 703–722. <https://doi.org/10.1177/0306312705055535>
- Cummings, J. N., & Kiesler, S. (2007). Coordination costs and project outcomes in multi-university collaborations. *Research Policy*, 36(10), 1620–1634. <https://doi.org/10.1016/j.respol.2007.09.001>
- Fecher, B., & Friesike, S. (2014). *Open Science: One Term, Five Schools of Thought* (S. Bartling & S. Friesike, Eds.; pp. 17–47). Springer International Publishing. https://doi.org/10.1007/978-3-319-00026-8_2
- Grant, S., Corker, K., Mellor, D., Stewart, S., Cashin, A., Lagisz, M., Mayo-Wilson, E., Moher, D., Umpierre, D., Barbour, V., Buck, S., Collins, G., Hazlett, H., Hrynaskiewicz, I., Lee, C., Parker, T., Rethlefsen, M., Toomey, E., & Nosek, B. (2025). *TOP 2025: An update to the transparency and openness promotion guidelines*. https://doi.org/10.31222/osf.io/nmfs6_v2
- Moshontz, H., Campbell, L., Ebersole, C. R., IJzerman, H., Urry, H. L., Forscher, P. S., Grahe, J. E., McCarthy, R. J., Musser, E. D., Antfolk, J., Castille, C. M., Evans, T. R., Fiedler, S., Flake, J. K., Forero, D. A., Janssen, S. M. J., Keene, J. R., Protzko, J., Aczel, B., ... Chartier, C. R. (2018). The Psychological Science Accelerator: Advancing Psychology Through a Distributed Collaborative Network. *Advances in Methods and Practices in Psychological Science*, 1(4), 501–515. <https://doi.org/10.1177/2515245918797607>
- Rep. Foxx, V. [R.-N. (2019). *H.R.150 - 116th Congress (2019-2020): GREAT Act*. <https://www.congress.gov/bill/116th-congress/house-bill/150>
- Rep. Ryan, P. D. [R.-W. (2019). *H.R.4174 - 115th Congress (2017-2018): Foundations for Evidence-Based Policymaking Act of 2018*. <https://www.congress.gov/bill/115th-congress/house-bill/4174>

- Swartz, T. H., Palermo, A.-G. S., Masur, S. K., & Aberg, J. A. (2019). The Science and Value of Diversity: Closing the Gaps in Our Understanding of Inclusion and Diversity. *The Journal of Infectious Diseases*, 220(220 Suppl 2), S33–S41. <https://doi.org/10.1093/infdis/jiz174>
- Wilkinson, M. D., Dumontier, M., Aalbersberg, Ij. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., Silva Santos, L. B. da, Bourne, P. E., Bouwman, J., Brookes, A. J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C. T., Finkers, R., ... Mons, B. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3(1), 160018. <https://doi.org/10.1038/sdata.2016.18>
- Wuchty, S., Jones, B. F., & Uzzi, B. (2007). The Increasing Dominance of Teams in Production of Knowledge. *Science*, 316(5827), 1036–1039. <https://doi.org/10.1126/science.1136099>